Prolog

The search tree

Prolog search tree

- The initial goal is at the root
- Each demonstration (resolution step) is a branch
- Each clause choice is a child node
  - The order (left-to-right) corresponds to the order of clause declaration in the program
- Leaves are:
  - If the new goal is empty
  - The current goal cannot be demonstrated

Prolog tree search

- Depth-first search
  - Based on the declaration order of the clauses
  - When the search fails, the tree is explored up to the next unexplored goal (backtracking)
- When a succesful leave is found
  - The solution is composed of all unifications applied between the root and this leave

The order

- of the literals in the body of a clause
- and of the order of the clauses

has an impact on the order the found solutions

A first example

```
f(a).
f(b).
g(a).
g(b).
h(b).
k(X):- f(X), g(X), h(X).

?- k(Y).
```

Example: Prolog search

```
parent(john, mary).
parent(mary, susan).
parent(john, paul).

ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z),
ancestor(Z, Y).
```

```
?- ancestor(john, susan).
```

Example: search tree

```
                ancestor(john, susan)
                /                   \
          (rule 1)                  (rule 2)
          /                           \
parent(john, susan)                 parent(john, Z), ancestor(Z, susan)
     |                                |
   FAIL                        Z = mary or Z = paul
                                /          \
                       ancestor(mary, susan)   ancestor(paul, susan)
                            |                     |
                      (rule 1 or 2)              FAIL
                            |
                   parent(mary, susan)
                            |
                          SUCCESS
```

Another example (3 versions)

```
father(charles,jean).
noble(henri).
noble(louis).
noble(charles).
noble(X):- father(Y,X), noble(Y).
```

```
father(charles,jean).
noble(X):- father(Y,X), noble(Y).
noble(henri).
noble(louis).
noble(charles).
```

```
father(charles,jean).
noble(henri).
noble(louis).
noble(charles).
noble(X):- noble(Y), father(Y,X).
```

```
?- noble(jean).
```

Yet another example

```
love(vincent,mimi).
love(marcel,mimi).
jealous(X,Y) :- love(X,Z), love(Y,Z).

?- jealous(X,Y).
```

How many solution? … 4

Arithmetic operator

- Prolog has integer and floating point values
  - + - *
  - / (floating-point division)
  - // (integer division)
  - mod
  - ** (exponentiation)
  - Sqrt ln

Arithmetic evaluation

- Prolog does not automatically evaluate arithmetic expressions
  - Evaluation not part of unification
- You must force the evaluation using the `is` operator

```
?- write(1+2).
1+2
true.

?- X is 1+2.
X = 3.

?- X = 1+2.
X = 1+2.

?- 1+2=3.
false.
```

```
?- 1+2 is 3.
false.

?- 3 is 1+2.
true.

?- 1+2=2+1.
false.

?- X is 1+2, Y is 2+1, X=Y.
X = Y, Y = 3.
```

Comparison operators

- These operators evaluates expressions
  - X =:= Y (equal)
  - X =\= Y (not equal)
  - X < Y
  - X =< Y
  - X > Y
  - X >= Y
  - To be valid, all variables must be instantiated.

Term comparison

- == succeed if two terms are exactly identical
  - No unification
- \== means not ==

```
?- X=3.
X = 3.

?- X==3.
false.

?- x==x.
true.
```

```
?- X\=Y.
false.

?- X\==Y.
true.

?- X=3,X=Y,X==Y.
X = Y, Y = 3.
```

Example

```
reigns(philippeIV,1285,1314).
reigns(philippeIII,1270,1385).

king(Name,Year):- reigns(Name,A,B),
                  Year>=A, Year=<B.
```

Example

```
min(X,Y,X) :- X<Y.
min(X,Y,Y) :- X>=Y.
```

Which queries are valid ?

Examples

```
gcd(U,0,U).
gcd(U,V,W) :- not(V=0), R is U mod V, gcd(V,R,W).

    fact(0, 1).
fact(N, F) :-
   N > 0, N1 is N - 1,
   fact(N1, F1), F is F1 * N.

    pow( X, 1, X ).
pow( X, Y, Z ) :-
   Y > 1, Y1 is Y - 1,
   pow( X, Y1, Z1 ), Z is X * Z1.
```

Roots of a quadratic equation

```
resolution2(A,B,C,X):- discriminant(A,B,C,Discriminant),
                           root(A,B,C,Discriminant,X).
discriminant(A,B,C,Discriminant):-Discriminant is B*B-4*A*C.

root(A,B,C,0,X):- X is -B/(2*A).
root(A,B,C,Discriminant,X):- Discriminant>0,
                         X is (B+sqrt(Discriminant))/(2*A).
root(A,B,C,Discriminant,X):- Discriminant>0,
                         X is (-B-sqrt(Discriminant))/(2*A).
```

Interval predicate

```
interval(K,L,H):- K>=L, K=<H.
```

Simple but not very useful…

Interval generator

```
interval(K,K,H):- K=<H.
interval(K,L,H):- L<H, L1 is L+1, interval(K,L1,H).

?- interval(X,3,6).
```

Find automorphic numbers

```
find_automorphic(K,L,H) :- interval(K,L,H),automorphic(K).

?- find_automorphic(K,3,500).
K = 5 ;
K = 6 ;
K = 25 ;
K = 76 ;
K = 376 ;
false.
```

Find automorphic numbers

```
count_digits(N, D) :-
    N < 10, !,
    D = 1.
count_digits(N, D) :-
    N >= 10,
    N1 is N // 10,
    count_digits(N1, D1),
    D is D1 + 1.

automorphic(N) :-
    N >= 0,
    count_digits(N, D),
    Power is 10 ** D,
    Square is N * N,
    Square mod Power =:= N.
```

Infinite generator

```
integer(0).
integer(N) :- integer(N1), N is N1+1.
```

- We need to find a way to stop it!